

Projet Technologies Web II / Programmation Web

Chat centralisé

I. Architecture du projet

Le *Projet_Chat* est organisé en plusieurs applications Django distinctes afin de séparer clairement les responsabilités côté serveur et faciliter la maintenance. L'arborescence principale du projet contient :

- *accounts/* : gestion des utilisateurs (inscription, connexion, déconnexion)
- *chat_messages/* : gestion des salons de discussion, messages et fonctionnalités liées au chat
- *Projet_Chat/* : configuration globale du projet, fichiers *settings.py*, *urls.py* et *wsgi.py*
- *templates/* et *static/* : fichiers HTML, CSS, et ressources statiques
- *db.sqlite3* : base de données SQLite utilisée pour stocker utilisateurs, messages et salons

Le routage des URLs est centralisé dans *Projet_Chat/urls.py* :

- La racine / redirige vers la page de connexion.
- L'URL */admin/* donne accès à l'interface d'administration Django.
- Les URLs commençant par */accounts/* sont gérées par l'app *accounts*.
- Les URLs liées aux salons, aux messages et à la modération sont gérées par l'app *chat_messages*.

A. App *accounts* : gestion des utilisateurs

L'application *accounts* est responsable de la gestion complète des utilisateurs pour le chat. Elle repose sur le modèle **User** intégré à Django et utilise des formulaires personnalisés basés sur *UserCreationForm* et *AuthenticationForm*.

- **Inscription (signup)** : création de compte avec nom d'utilisateur, adresse email, et mot de passe avec confirmation.
- **Connexion (login)** : connexion pour les utilisateurs existants via nom d'utilisateur et mot de passe. La validation est gérée par Django, assurant la sécurité.
- **Déconnexion (logout)** : permet de mettre fin à la session de l'utilisateur en toute sécurité.
- **Templates et interface** : Les pages *login.html* et *signup.html* utilisent `{% csrf_token %}` pour la sécurité. Les formulaires sont rendus avec `{{ form.as_p }}` pour générer automatiquement les champs et messages d'erreur.
- **Sécurité et bonnes pratiques** : L'app suit les principes de Django Auth, protège contre les attaques CSRF et garantit que les mots de passe sont stockés de manière sécurisée (hachage). Les formulaires sont également validés côté serveur pour éviter toute manipulation.

En résumé, *accounts* fournit une base solide et sécurisée pour gérer les utilisateurs, sur laquelle les fonctionnalités du chat (messaging, salons, droits) peuvent s'appuyer.

B. App *chat_messages* : gestion des salons et des messages

L'application *chat_messages* gère l'ensemble des fonctionnalités liées au chat. Elle repose sur deux modèles principaux :

- **Salon** : représente un salon de discussion. Chaque salon possède un nom, un administrateur, une liste de membres et éventuellement des utilisateurs bannis. Les permissions sont également définies pour permettre la modération, comme l'exclusion d'un membre.
- **Message** : représente un message envoyé dans un salon. Chaque message est lié à un salon et à un expéditeur, et contient le texte et la date d'envoi.

Les vues de l'application permettent :

- **La création, l'affichage et la suppression de salons.** Seul l'administrateur, créateur du salon, peut supprimer un salon ou exclure un membre.
- **La gestion des membres** : les utilisateurs peuvent rejoindre un salon, et les administrateurs peuvent exclure des membres.
- **L'envoi et l'affichage des messages** : les messages sont affichés dans l'ordre chronologique et incluent le nom de l'expéditeur et l'heure d'envoi.

L'app intègre également la gestion des emojis via le widget *EmojiPickerTextInput*, permettant aux utilisateurs d'ajouter facilement des emojis dans leurs messages.

Enfin, les templates HTML sont conçus pour être clairs et interactifs :

- *salon_list.html* affiche tous les salons disponibles.
- *create_salon.html* permet de créer un nouveau salon.
- *message.html* gère l'affichage du chat, des membres et des options de modération.

II. Fonctionnalité de temps réel

Afin d'améliorer l'expérience utilisateur, une fonctionnalité de rafraîchissement dynamique des messages a été mise en place. L'application repose sur une approche AJAX : une vue Django dédiée (*get_messages_json*) renvoie les messages d'un salon au format JSON avec l'expéditeur, le contenu et l'heure d'envoi, tandis que le front-end, à l'aide de Javascript, interroge régulièrement le serveur afin de mettre à jour la fenêtre de discussion sans recharger la page.

Côté client, le fichier *main.js* effectue les opérations suivantes :

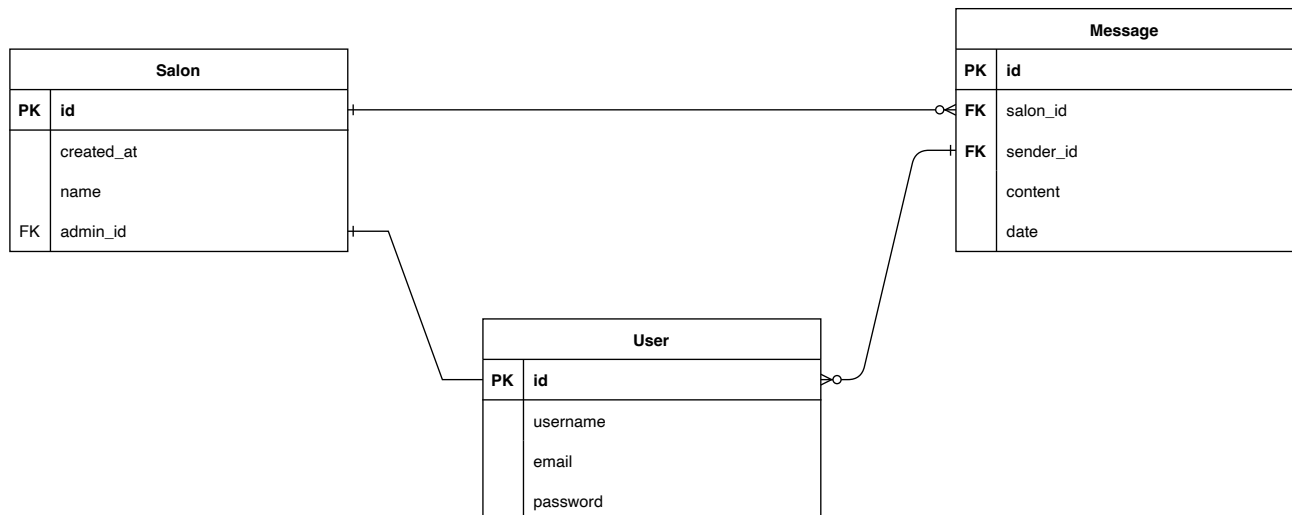
1. Sélection du salon et de l'utilisateur courant à partir des attributs *data-salon-id* et *data-username*.
2. Récupération initiale des messages via fetch vers l'URL JSON du salon.
3. Rendu dynamique des messages : chaque message est inséré dans la fenêtre de chat, avec une distinction visuelle entre les messages de l'utilisateur courant et ceux des autres.
4. Rafraîchissement automatique : le script appelle la vue JSON toutes les secondes (*setInterval*) pour récupérer les nouveaux messages et les afficher sans recharger la page.
5. Maintien du scroll : si l'utilisateur est en bas du chat, la fenêtre défile automatiquement pour afficher les nouveaux messages.

Cette approche simple et efficace permet un chat interactif, fluide et réactif, tout en restant compatible avec les fonctionnalités de modération et de gestion des salons. Elle pourrait être améliorée ultérieurement avec des WebSockets pour un vrai temps réel instantané.

III. Modèles et base de données

La base de données du projet repose sur les trois modèles **User**, **Salon** et **Message** dont le détail est plus haut dans le rapport. Le schéma montre les relations entre ces entités :

- Un salon peut avoir plusieurs membres et plusieurs messages.
- Un utilisateur peut appartenir à plusieurs salons et envoyer plusieurs messages.
- Les permissions spécifiques, comme l'exclusion d'un membre, sont associées à chaque salon pour gérer la modération.



IV. Conclusion

Ce projet a permis de mettre en œuvre une application web complète en utilisant Django et Javascript, tout en respectant une architecture claire et sécurisée. Les fonctionnalités principales attendues (authentification, salons, messagerie, droits et modération) ont été implémentées avec succès. Le chat en quasi-temps réel offre une expérience utilisateur fluide. Le projet pourrait être amélioré à l'avenir par l'utilisation de WebSockets, l'ajout de notifications et l'utilisation de Bootstrap par exemple afin de le rendre plus beau.